

Blatt: Reguläre Sprachen

A1.1: Sprachen von regulären Ausdrücken

Der gegebene Ausdruck $a + a(a + b)^*a$ ist am besten zuerst in drei Abschnitte einzuteilen, um die Beschreibung zu vereinfachen, erstens " $a + a(\dots)$ ", zweitens " $(a + b)^*$ " und drittens " a ".

- Der Ausdruck $a + a(\dots)$, beschreibt die Vereinigung von zwei Sprachen, zum einen Die Sprache $L_1 = a$ und $L_2 = a(\dots)$, Dies bedeutet, dass ein Wort dieser Sprache entweder Teil von L_1 oder L_2 ist. Kurz gesagt ein entstehendes Wort ist entweder nur " a " (L_1) oder " $a\dots$ " (L_2)
- " $(a + b)^*$ " ist Teil von L_2 und besagt, dass eine endliche Anzahl, mal " $a + b$ " ausgeführt wird. " $a + b$ " ist hierbei eine Vereinigung und besagt, dass entweder a oder b ausgegeben wird. Zusammen bedeutet dies also, dass " $(a + b)^*$ " eine endliche Menge von hintereinander angereihten " a " und " b " ist.
- " a " bedeutet hier, dass das Wort immer mit einem " a " endet

Zusammenfassen ist also zu sagen, dass Wörter der beschriebenen Sprache, von " $a + a(a + b)^*a$ " entweder " a " oder ein Wort, welches mit " a " beginnt und endet und welches zwischen diesen beiden " a " eine endliche Reihe an " a " und " b " besitzt.

Mögliche Wörter:

- a
- $abba$
- $abaa$
- aa
- $aaaaa$
- $abbbaba$

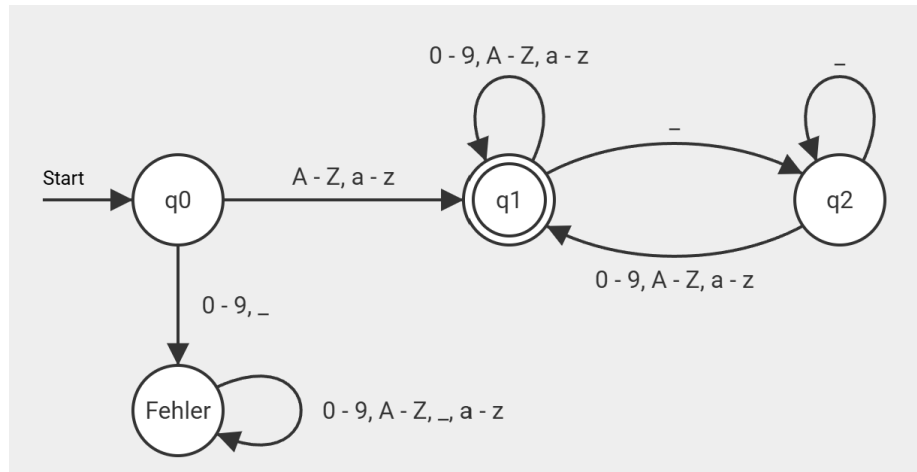
1 A1.2: Bezeichner in Programmiersprachen

Der reguläre Ausdruck, welcher die beschriebene Programmiersprache darstellt sieht, wie folgt aus:

$$(a - z + A - Z)(a - z + A - Z + 0 - 9 + _)* (a - z + A - Z + 0 - 9)$$

Hier ist anzumerken, dass die Speziellen Regeln bezüglich des Anfangsbuchstabens für Variablen und Parameter. Diese Regeln können in der Darstellung der Sprache nämlich vernachlässigt werden, weil die Tatsache, ob das Wort ein Parameter oder eine Variable ist, keine Auswirkung auf den Rest des Wortes hat.

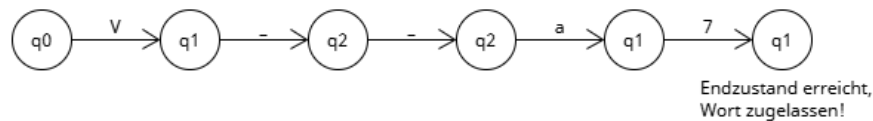
Die Darstellung der Sprache als DFA sieht, wie folgt aus:



Wörter, welche nicht mit einem Buchstaben anfangen, werden hier in einen Fehlerzustand gepackt, sodass keine Falschen Wörter angenommen werden können.

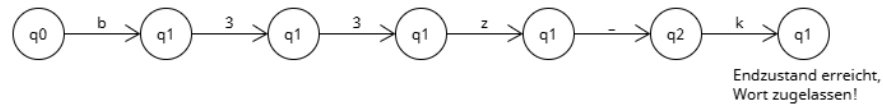
Hier werden die Zustandsübergänge für das Eingabewort "V_a7" dargestellt:

Eingegebenes Wort:
V_a7



Hier werden die Zustandsübergänge für das Eingabewort "b33_k" dargestellt:

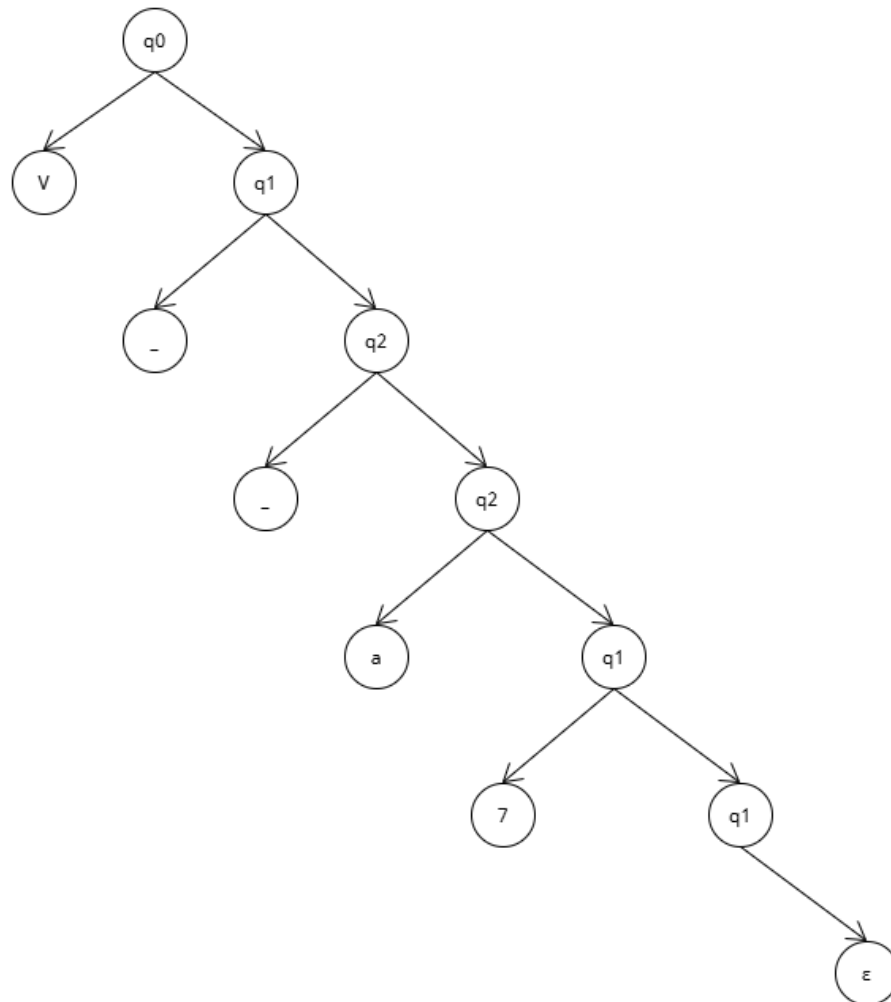
Eingegebenes Wort:
b33z_k



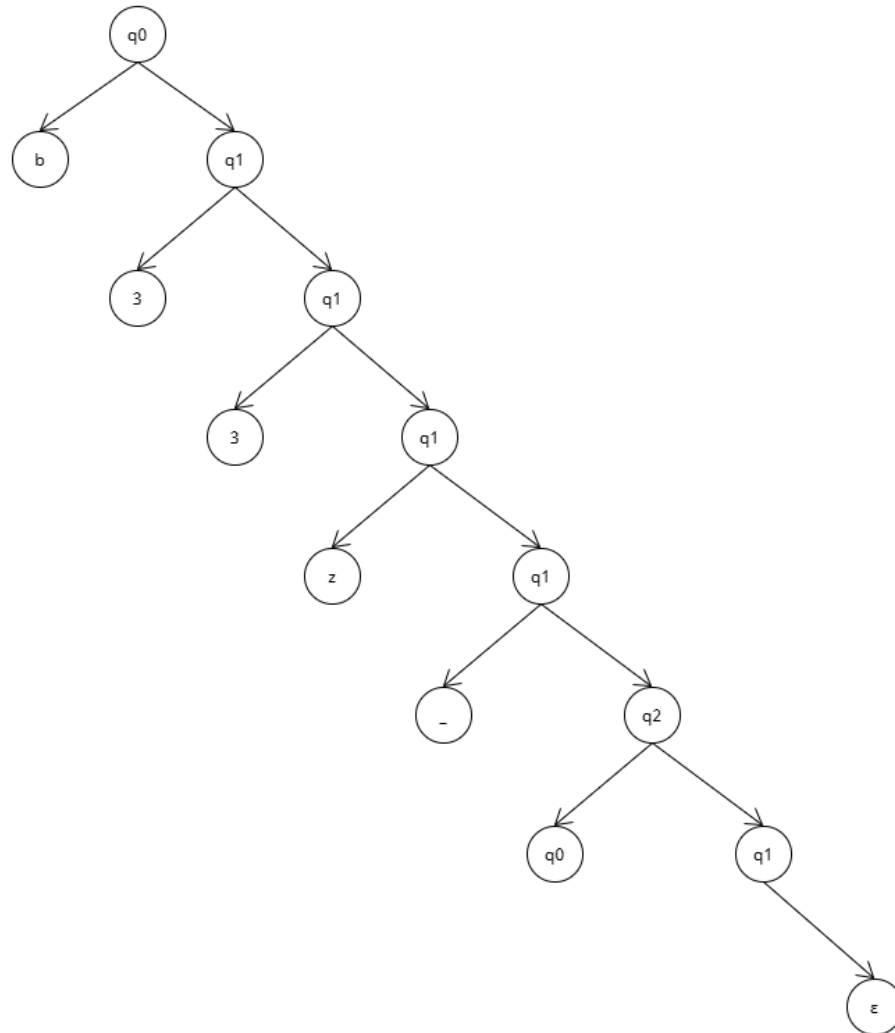
Die Grammatik für diese Sprache wird , wie folgt dargestellt:

$G = ($
 $N = \{q0, q1, q2\}$
 $T = \{a-z, A-Z, 0-9, _\}$
 $S = \{q0\}$
 $P = \{$
 $q0 \rightarrow (a-z)q1 \parallel (A-Z)q1$
 $q1 \rightarrow (a-z)q1 \parallel (A-Z)q1 \parallel (0-9)q1 \parallel _q2 \parallel \epsilon$
 $q2 \rightarrow (a-z)q1 \parallel (A-Z)q1 \parallel (0-9)q1 \parallel _q2$
 $\}$
 $)$

Der Ableitungsbaum für das Wort "V_a7" sieht, wie folgt aus:



Der Ableitungsbaum für das Wort "b33z_k" sieht, wie folgt aus:



A1.4: Mailadressen?

Die Aussage " $(a - z)^+ @ (a - z) . (a - z)$ " ist Ungültig für das erkennen von E-Mail Adressen aufgrund dessen, dass es vor und nach dem "." jeweils nur einen Buchstaben erkennen kann. Dies ist nicht der Schreibweise von " $a - z$ " verschuldet, weil hier selbst die Korrekte Darstellung durch " $a + b + c + \dots + z$ " noch nicht akkurat E-Mail Adressen einsehen könnte.

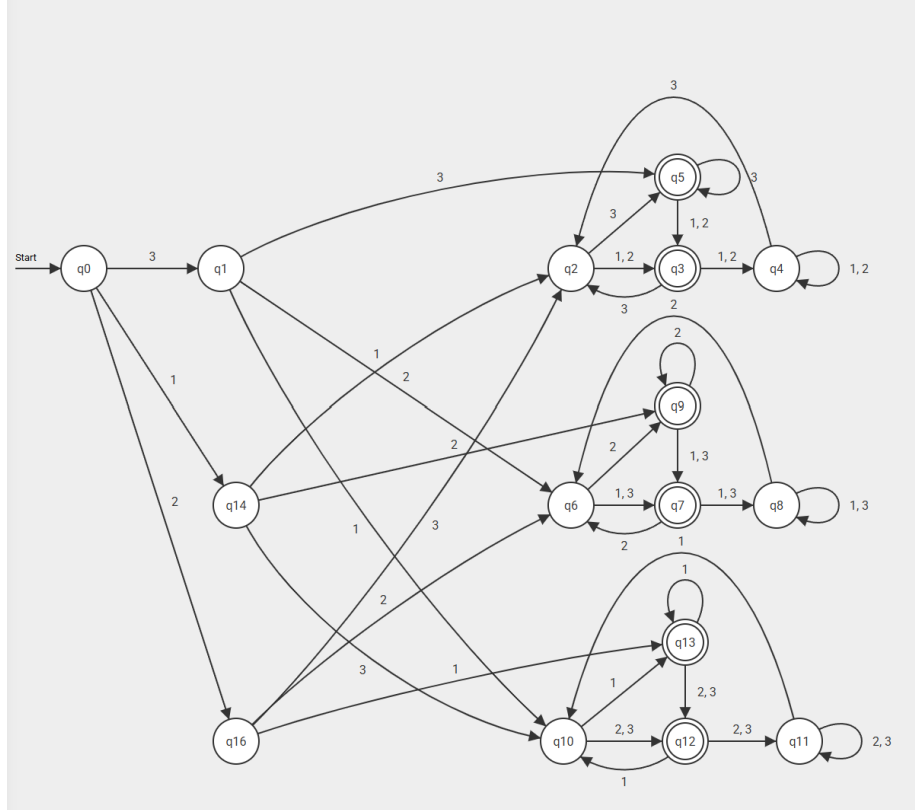
Mit $L_1 = a+b+c+\dots+z$, $L_2 = A+B+C+\dots+Z$ und $L_3 = 1+2+3+\dots+9$, würde die richtige Schreibweise, wie folgt aussehen:

$$(L_1 + L_2 + L_3)^+ @ (L_1 + L_2)^+ . (L_1 + L_2)^+$$

Die Änderung, die gemacht wurde, war das Ändern von den Ausdrücken " $(a - z)$ ", zu " $(a + b + c + \dots + z)$ " und das hinzufügen von " $(\dots)^+$ " an die Klammern vor und hinter dem ".". Hierdurch kann die Sprache nun beliebig viele Buchstaben für den Namen der E-Mail (z.B. "outlook", gmail, hsbj) und den Namen der genutzten Domäne (z.B. "de", "com", "tv") annehmen, anstatt nur einem Buchstaben, wie zuvor. Zudem wurde die Menge an verfügbaren Zeichen erweitert, dass auch Großschreibung berücksichtigt wird und die verfügbaren Zeichen für den Namen der Adresse (z.B. MaxMusterman01), wurde um Ziffern erweitert.

A1.5: Der zweitletzte Buchstabe

Der benötigte Automat sieht, wie folgt aus:



A1.6: Sprache einer regulären Grammatik

Die gegebene Grammatik kann entweder, als folgender regulärer Ausdruck dargestellt werden:

$$a((b+c)^+dc^+)(a+b)$$

Oder als der folgende DFA:

